

Token Efficiency, Not Price, Determines the Cost of Reliable Structured Output

Hongyu Chen
AgentSystem Research Lab

Abstract

When deploying LLMs for structured-output tasks, the prevailing cost heuristic is simple: use the cheapest model per token that works. We show this heuristic is systematically wrong. Across 9 models spanning 7 local open-source deployments (4.7B–30B parameters) and 2 commercial APIs, we find that **token efficiency—how many tokens a model needs to produce valid JSON—dominates price-per-token in determining actual cost per correct output**. DeepSeek V4, at $4\times$ the unit price of MiniMax M3, is $3\text{--}7\times$ cheaper per correct governance output because it uses $5\times$ fewer tokens per call. Three local open-source models (HY-MT2-7B, GPT-OSS-20B, Nemotron-30B) achieve 100% compliance across all governance schemas at zero API cost—on consumer GPU hardware, 2.9s latency—with HY-MT2-7B strictly dominating on all single-call metrics. The widely advocated “delegation” strategy—assigning reasoning to a cheap model and formatting to an expensive one—costs $7.5\times$ more than calling the expensive model directly, making it an economic anti-pattern when both models can format correctly. We provide a portable, SSC-driven decision rule for model selection that depends only on per-model token efficiency and pass rate—quantities measurable with 12 calibration calls per model—and generalizes across price changes.

1 Introduction

When deploying language models for structured-output tasks—JSON generation, API call formatting, governance verdicts—the default cost heuristic is simple: *use the cheapest model per token that works reliably*. If MiniMax M3 costs \$0.07 per million input tokens and DeepSeek V4 costs \$0.28, the intuition is clear: MiniMax is $4\times$ cheaper. Delegate formatting to MiniMax; reserve DeepSeek for tasks MiniMax cannot handle. This intuition underpins a growing body of work on cost-efficient model routing and multi-model delegation [1, 5, 6].

We show this heuristic is systematically wrong.

The flaw is not in the arithmetic of price-per-token. It is in the *implicit assumption that all models consume comparable numbers of tokens to produce the same output*. Across 9 models spanning local open-source deployments and commercial APIs, we find that token consumption per correct JSON output varies by a factor of $20\times$ (37 to 800 tokens). This variation dwarfs the $4\times$ price difference between the cheapest and most expensive model. The result is a **cost inversion**: the “expensive” API is often significantly cheaper per correct output than the “cheap” one.

Three findings structure this paper:

- F1: Token efficiency dominates price.** Within the API tier, DeepSeek V4 (expensive per token, efficient per call) is $3\text{--}7\times$ cheaper per correct governance output than MiniMax M3 (cheap per token, inefficient per call). This inversion holds across all four tested governance roles and widens with schema complexity.
- F2: The delegation anti-pattern.** Assigning reasoning to MiniMax and formatting to DeepSeek—a strategy widely advocated for cost reduction [1]—costs $7.5\times$ more than calling DeepSeek directly. The coordination overhead of two API calls and MiniMax’s token

inefficiency combine to make delegation economically harmful when both models can format correctly. Delegation only pays off when a *formatting capability gap* exists—not a price gap.

F3: The zero-cost local tier. Three local open-source models (HY-MT2-7B, GPT-OSS-20B, Nemotron-30B) achieve 100% compliance across all four governance roles at zero API cost, 2–5 second latency, on consumer GPU hardware. HY-MT2-7B strictly dominates the other two on all single-call metrics. For organizations running governance pipelines at scale, this eliminates the need for cloud APIs entirely on schemas with $SSC \leq 8.5$.

These findings converge on a practical decision rule: when choosing models for structured-output pipelines, **optimize for token efficiency, not price per token**. The rule is portable: it depends on per-model quantities (pass rate, average tokens per call) that can be measured with 12 calibration calls, and it survives changes in API pricing because it is expressed as a function of the price ratio rather than absolute prices.

2 Experimental Design

2.1 Models

We test 9 models spanning three tiers (Table 1): 7 local open-source models running via llama.cpp with Vulkan GPU acceleration, and 2 commercial APIs. Local models use 4–8 bit quantization (GGUF format); API models use provider default configurations.

Table 1: Models Tested

Model	Type	Size	\$/1M Input/Output
HY-MT2-7B (Q8_0)	Local GGUF	7B	0 / 0
GLM-4.7-Flash (Q5_K_M)	Local GGUF	4.7B	0 / 0
DeepSeek-R1-Qwen3-8B (BF16)	Local GGUF	8B	0 / 0
GPT-OSS-20B (MXFP4)	Local GGUF	20B	0 / 0
Gemma-4-26B (Q4_0)	Local GGUF	26B	0 / 0
Qwen3.6-27B (Q5_K_M)	Local GGUF	27B	0 / 0
Nemotron-30B (Q4_K_M)	Local GGUF	30B	0 / 0
DeepSeek V4	Cloud API	—	0.28 / 1.10
MiniMax M3	Cloud API	—	0.07 / 0.50

2.2 Governance Roles as Standardized Benchmarks

We use four governance roles from an agent harness pipeline [2] as standardized structured-output benchmarks (Table 2). Each role has a fixed JSON output schema, identical prompts across all models, and temperature $T = 0.0$.

Table 2: Governance Roles and SSC Scores

Role	SSC	Type	Schema Structure
LoopDetection (LD)	1.4	flat	4 fields
PlanInterrogation (PI)	2.4	string arrays	3 arrays
ReflectionCheck (RC)	4.0	nested	4 fields + string array
ErrorClassifier (EC)	8.5	array-of-objects	object array

SSC intuition. SSC (Schema Structure Complexity) combines nesting depth, array-of-objects presence, and field count into a single score [2]. For this paper, the only necessary

intuition: LD (1.4) is a simple flat JSON object; EC (8.5) is a deeply nested array of objects. Higher SSC = structurally harder to format.

2.3 Metrics and Protocol

- **Pass rate:** proportion of trials producing syntactically valid JSON (extracted from model output, validated via `json.loads`).
- **Token efficiency:** completion tokens per call—lower values indicate the model produces JSON more concisely.
- **Cost per correct output:** $(\text{prompt_tokens} \times p_{\text{in}} + \text{completion_tokens} \times p_{\text{out}}) / \text{pass_rate}$, where p denotes price per token.

Protocol. All experiments use $T = 0.0$, `max_tokens=800`, rich contextual prompts (full task description, schema specification, scenario context). API models: $n = 5$ trials per cell (DeepSeek, MiniMax) or $n = 10$ (retry experiments). Local models: $n = 3$ trials per cell, llama-server Vulkan backend, `ctx_size=2048`.

Retry protocol (MM-retry-3). Simple retry without budget escalation: each attempt uses the identical prompt and `max_tokens=800`. Maximum 3 attempts; if all fail, the call is counted as a failure. This is a conservative design—budget-escalation retry would raise, not lower, expected cost.

3 The Token-Efficiency Surprise

3.1 The Intuitive (but Wrong) Cost Model

Table 1 shows MiniMax costs $4\times$ less per token than DeepSeek (\$0.07 vs. \$0.28 per 1M input tokens). The naive cost model predicts MiniMax should be approximately $4\times$ cheaper per governance call.

3.2 The Actual Cost Data

Table 3 shows the reality.

Table 3: Cost per Correct Output: DeepSeek vs. MiniMax

Role (SSC)	DS Pass	DS Cost/Correct	MM Pass	MM Cost/Correct
EC (8.5)	5/5 (100%)	\$0.00007	5/5 (100%)	\$0.00026
RC (4.0)	5/5 (100%)	\$0.00011	5/5 (100%)	\$0.00021
PI (2.4)	5/5 (100%)	\$0.00010	5/5 (100%)	\$0.00036
LD (1.4)	5/5 (100%)	\$0.00006	5/5 (100%)	\$0.00012

DeepSeek is **2–4 $\times$ cheaper per correct output** than MiniMax across all four roles, despite being $4\times$ more expensive per token. Both models achieve 100% pass rates under the 800-token budget—the cost difference comes entirely from token consumption.

3.3 Why: Token Efficiency, Not Price

MiniMax consumes 4–6 $\times$ more tokens per call than DeepSeek (Table 4). The model’s per-token price advantage is consumed—and inverted—by its token inefficiency.

Define the portable metric:

Table 4: Token Consumption per Call (Total = Prompt + Completion)

Role	DS Total Tokens	MM Total Tokens	Ratio
EC (8.5)	138 (95+43)	738 (264+474)	5.3×
RC (4.0)	170 (92+78)	648 (258+390)	3.8×
PI (2.4)	171 (102+69)	946 (265+681)	5.5×
LD (1.4)	114 (86+28)	464 (253+211)	4.1×

Parentheses show (prompt_tokens + completion_tokens). Cost uses the full formula:
 $(\text{prompt_tokens} \times p_{\text{in}} + \text{completion_tokens} \times p_{\text{out}})/10^6$.

$$\text{Cost-per-Correct} = \frac{\bar{t}_{\text{prompt}} \cdot p_{\text{in}} + \bar{t}_{\text{completion}} \cdot p_{\text{out}}}{\text{pass_rate}}$$

where $\bar{t}$ denotes average tokens and p denotes price per token. This metric is computable for any model pair and any task. The cost inversion occurs when:

$$\frac{\bar{t}_{\text{cheap}}}{\bar{t}_{\text{expensive}}} > \frac{p_{\text{expensive}}}{p_{\text{cheap}}}$$

For MiniMax vs. DeepSeek: token ratio = 5.3 > price ratio = 4.0. The inequality holds—MiniMax’s inefficiency outweighs its price advantage. This condition generalizes: any model whose token consumption exceeds the price ratio of the competition will be more expensive per correct output, regardless of how cheap it is per token.

4 The Delegation Anti-Pattern

A widely advocated strategy for cost reduction is *delegation*: assign reasoning to a cheap model and formatting to a capable one [1]. The intuition is that the expensive model only runs for a brief formatting step, saving on output tokens.

For delegation and retry experiments, we increased to $n = 10$ trials per condition (vs. $n = 5$ in §3) to improve statistical confidence for the cost comparison. Table 5 shows this intuition fails for the MiniMax–DeepSeek pair.

Table 5: Delegation Cost Analysis (EC Role, SSC=8.5, $n=10$)

Strategy	Pass Rate	Avg Cost/Call	Cost/Correct
DS-native	10/10 (100%)	\$0.00007	\$0.00007
MM-retry-3	9/10 (90%)	\$0.00049	\$0.00055
MM→DS delegate	10/10 (100%)	\$0.00052	\$0.00052

MM→DS delegation costs **7.5×** more than DS-native. The reason is structural: delegation requires two API calls (reasoning + formatting). MiniMax’s token inefficiency propagates into the reasoning phase, producing ~ 400 tokens of reasoning text that DeepSeek must re-read as input (billed at \$0.28/1M). The coordination overhead exceeds any savings from the formatting step.

When does delegation pay off? Delegation is economically justified only when a *formatting capability gap* exists—i.e., the reasoning model literally cannot produce valid JSON. If both models can format correctly (as our data shows for current-generation MiniMax), delegation is a pure cost penalty. This refines prior delegation heuristics from “always delegate” to “delegate across capability gaps, not price gaps.”

5 The Zero-Cost Local Tier

5.1 The 9-Model SSC Profile

Table 6 shows the complete SSC profile across all 9 models. This is the central empirical contribution of the paper.

Table 6: 9-Model SSC Profile (Rich Prompts, max_tokens=800)

Model	Size	EC (8.5)	RC (4.0)	PI (2.4)	LD (1.4)
<i>Local GGUF</i>					
HY-MT2-7B	7B	3/3 (37t)	3/3 (46t)	3/3 (44t)	3/3 (26t)
GPT-OSS-20B	20B	3/3 (86t)	3/3 (292t)	3/3 (400t)	3/3 (91t)
Nemotron-30B	30B	3/3 (465t)	3/3 (609t)	3/3 (480t)	3/3 (202t)
Gemma-4-26B	26B	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)	3/3 (612t)
DeepSeek-R1-8B	8B	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)
Qwen3.6-27B	27B	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)
GLM-4.7-Flash	4.7B	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)	0/3 ($\geq 800t$)
<i>Cloud API</i>					
DeepSeek V4	—	5/5 (138t)	5/5 (170t)	5/5 (171t)	5/5 (114t)
MiniMax M3	—	10/10 (738t)	10/10 (648t)	10/10 (946t)	10/10 (464t)

All $\geq 800t$ entries are `finish_reason=length`: the model reached the 800-token ceiling before completing JSON. The reported count is the truncation threshold, not the tokens the model would have consumed to finish.

Three findings emerge from this matrix.

Finding 1: Three local models achieve 100% SSC coverage. HY-MT2-7B, GPT-OSS-20B, and Nemotron-30B pass all four governance roles—including EC (SSC=8.5), the hardest schema. At zero API cost, these models eliminate the need for cloud APIs for any governance task with $SSC \leq 8.5$.

Finding 2: All failures are token-budget failures. Every single failure across 84 local-model trials had `finish_reason=length`—the model consumed all 800 tokens before completing JSON. Zero models produced *malformed* JSON. The four failing models are not “incapable of structured output”—they need more than 800 tokens to finish. This is a budget allocation problem, not a capability deficit.

Finding 3: DeepSeek-R1 is a reasoning-model outlier. Its 0/12 pass rate—including on the simplest schema (LD, 4 flat fields)—is not due to verbosity but to `<think>` tag emission: internal reasoning tokens consume the entire 800-token budget before any visible JSON is generated. Unlike the other failing models, which produce partial JSON before truncation, DeepSeek-R1 produces zero output tokens across all schemas—the budget is fully consumed by the reasoning trace. This is the same mechanism that affected earlier versions of other reasoning models [2], and it represents the limiting case of token-budget competition.

5.2 HY-MT2-7B Strictly Dominates the Local Tier

Even within the passing local models, one choice is unambiguous (Table 7).

Table 7: Local Model Single-Call Comparison

Model	Tokens/Call	t/s	Wall Time	Dominated?
HY-MT2-7B	37	13	2.9s	No—Pareto-optimal
GPT-OSS-20B	217	49	4.4s	Yes
Nemotron-30B	439	99	4.4s	Yes

HY-MT2-7B is **strictly dominant** on all three single-call metrics: lowest wall time (2.9s), fewest tokens (37), and zero API cost. Nemotron’s $7.6\times$ higher token-per-second rate (99 vs. 13) does not translate to lower latency because it generates $12\times$ more tokens per call—its speed advantage is consumed by verbosity. GPT-OSS-20B occupies an intermediate position but is also dominated.

Batch throughput under shared context may favor larger models (higher GPU utilization across parallel calls), but verifying this requires batch inference benchmarks left to future work. For the single-call scenarios typical of governance pipelines, HY-MT2-7B is the unambiguous choice within the zero-cost tier.

6 A Practical Routing Framework

The findings converge on a simple, data-driven routing rule (Figure 1).

Step 1: SSC-based tier selection.

If $\text{SSC} < 8.5$ AND a local model is available:

→ **HY-MT2-7B** (Pareto-optimal: \$0 cost, 2.9s latency, 100% pass)
(GPT-OSS-20B and Nemotron-30B are strictly dominated on single-call metrics)

Otherwise ($\text{SSC} \geq 8.5$ or no local model):

Step 2: Check for formatting-capability gap.

If the reasoning model *cannot* produce valid JSON:

→ **Delegate** (reasoner → formatter across capability gap)

If the reasoning model *can* produce valid JSON:

→ **DeepSeek-native** (\$0.00007/call, most token-efficient API)

Portable threshold: $\frac{\bar{t}_{\text{cheap}}}{\bar{t}_{\text{expensive}}} < \frac{p_{\text{expensive}}}{p_{\text{cheap}}}$

Figure 1: Routing Decision Rule Based on SSC and Token Efficiency

Generalizability. The routing decision depends on three measurable per-model quantities—SSC pass profile, average tokens per call, and price per token—none specific to our model set. Any new model can be profiled with 12 calibration calls (4 roles $\times$ 3 trials). The decision threshold is expressed as a function of the model pair’s token-efficiency ratio and price ratio:

$$\text{Use cheaper model} \iff \frac{\bar{t}_{\text{cheap}}}{\bar{t}_{\text{expensive}}} < \frac{p_{\text{expensive}}}{p_{\text{cheap}}}$$

When API prices change, the threshold shifts predictably. The method survives price fluctuations because it depends on the *ratio*, not absolute values.

Cache hit rate as upside. Our cost calculations assume worst-case (cache miss on every call). In production, consecutive calls to the same model with identical system prompts achieve prompt cache hit rates of 40–80% [1], reducing actual cost by a proportional factor. The SSC decision threshold persists under caching because it depends on the *relative* cost between models, not the *absolute* cost.

7 Related Work

Structured-output reliability. Constrained decoding (Guidance [3], Outlines [4], LMQL) and API-level JSON mode enforce output schemas at the token or sampler level. These ap-

proaches focus on *how* to enforce formatting; we focus on *at what cost*—showing that the cheapest enforcement strategy depends on per-model token efficiency, not per-token price.

Model delegation and multi-agent systems. Prior work advocates delegating reasoning and formatting to different models as a general reliability strategy [2]. Fable 5’s cost-efficient harness [1] established the qualitative insight that frontier models can orchestrate cheaper workers. Our work extends this by providing a **quantitative boundary condition**: delegation is only cost-effective when the worker’s per-task token consumption does not exceed the orchestrator’s by more than a factor of the price ratio. For the MiniMax–DeepSeek pair, the worker’s $5\times$ token inefficiency inverts this boundary, making delegation an economic anti-pattern. **Our findings do not invalidate delegation; they define when it pays off.**

API cost optimization and LLM routing. Recent work on model routing (Frugal-GPT [5], RouteLLM [6]) optimizes cost by selecting among models based on predicted task difficulty. These approaches use learned routers trained on task-specific data. Our framework is complementary: it uses a static, pre-computed per-model profile (SSC tolerance + token efficiency) requiring only 12 calibration calls per model, and generalizes across any model without task-specific training.

Open-source model evaluation. Standard benchmarks (MMLU, HumanEval, HELM) evaluate models on task accuracy. Our work evaluates a different construct: *structured-output token efficiency*—the number of tokens a model consumes to produce valid JSON. This dimension has not been systematically measured across model families and is directly relevant to deployment cost.

8 Limitations and Future Work

Model recency. API model behavior changes over time (MiniMax improved its pass rate from our earlier measurements). SSC profiles should be periodically recalibrated—a process requiring only 12 calls per model.

Local model coverage. Our 7 local models all use GGUF quantization. Full-precision or differently quantized inference may yield different token-efficiency profiles. Batch inference benchmarks are needed to assess whether larger local models (Nemotron, GPT-OSS) gain advantages under concurrent load.

Task generalization. Governance roles provide a controlled testbed with clear SSC variation. Generalization to code generation, data extraction, and other structured-output domains requires separate validation, though the portable cost-per-correct metric applies identically.

Price sensitivity. The routing decision rule is a function of the price ratio, not absolute prices. When API prices change, the threshold shifts predictably. We provide the computation method, not a fixed recommendation tied to current pricing.

9 Conclusion

When choosing models for structured-output pipelines, the dominant factor in actual cost is not the price per token—it is how many tokens the model needs to succeed. Token efficiency can invert the apparent cost ordering: the “expensive” API is often the cheapest per correct output. Three local open-source models achieve 100% compliance at zero API cost, with HY-MT2-7B strictly dominating on all single-call metrics. Delegation—widely advocated as a cost-saving pattern—proves to be an economic anti-pattern when the reasoning model can format correctly, costing $7.5\times$ more than direct API calls. The practical implication is a portable, SSC-driven routing rule based on two measurable quantities: per-model token efficiency and pass rate. Any new model can be profiled with 12 calibration calls and plugged into the framework—a process that survives API price changes and model updates.

Data Availability

All experiment data, analysis scripts, and model outputs are available at <https://github.com/LuisChen1Q84/ssc-paper>. Local model inference uses llama.cpp with Vulkan GPU acceleration; API calls use provider standard endpoints.

References

- [1] Anthropic. *Claude Fable 5 Cost-Efficient Harness Design*. Technical Report, 2026.
- [2] Chen, H. *Empirical Findings for Reliable Structured Output in Multi-Model Agent Pipelines*. AgentSystem Research Lab, 2026.
- [3] Guidance. *Constrained decoding for LLMs*. 2023.
- [4] Willard, B.T. *Efficient Guided Generation for LLMs*. arXiv, 2023.
- [5] Chen, L., Zaharia, M., and Zou, J. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. arXiv:2305.05176, 2023.
- [6] Ong, I. et al. *RouteLLM: Learning to Route LLMs with Preference Data*. arXiv:2406.18665, 2024.
- [7] Park, J.S. et al. *Generative Agents: Interactive Simulacra of Human Behavior*. UIST, 2023.
- [8] Wu, Q. et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv, 2023.
- [9] LangChain. *Building applications with LLMs through composability*. 2023.
- [10] Zhou, Y. et al. *Large Language Models Are Human-Level Prompt Engineers*. ICLR, 2023.
- [11] Zheng, L. et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS, 2023.
- [12] Liang, P. et al. *Holistic Evaluation of Language Models*. TMLR, 2023.